Static Feature Specification

Honeypot Detection Project — Step 6

Pipeline 1: Static Analysis | Feature Engineering Reference

Overview

These 10 features are extracted from each `.sol` contract file and fed into the Random Forest classifier. Each feature targets a known honeypot deception pattern from the HoneyBadger taxonomy (Torres et al., 2019).

Feature Definitions

Feature 1: `uses_tx_origin`

- **Type:** Boolean
- **What it checks:** Does the contract use `tx.origin` for access control or authentication?
- **Why it matters:** `tx.origin` returns the original transaction sender, not the immediate caller. Honeypot owners use this to silently block anyone except themselves from withdrawing — victims can deposit but can never get their money back.
- **HoneyBadger type:** Hidden Transfer, Balance Disorder
- **Extraction method:** Regex or AST scan for `tx.origin`

Feature 2: `has_fallback`

- **Type:** Boolean
- **What it checks:** Does the contract define a fallback function (unnamed function or `receive()`)?
- **Why it matters:** The fallback runs when ETH is sent with no matching function call. Honeypots use this as the "deposit trap" — it accepts ETH silently while hiding the conditions that prevent withdrawal.
- **HoneyBadger type:** Balance Disorder, Hidden Transfer
- **Extraction method:** AST — look for `FunctionDefinition` nodes with no name, or named `receive`/`fallback`

Feature 3: `fallback_reverts_non_owner`

- **Type:** Boolean
- **What it checks:** Does the fallback function contain logic that reverts (or throws) for any caller who is not the owner?
- **Why it matters:** This is the core "trap" mechanism — the fallback appears to accept ETH from anyone, but secretly only the owner can actually trigger it successfully. Everyone else gets reverted after sending funds.
- **HoneyBadger type:** Balance Disorder
- **Extraction method:** AST — check if fallback body contains `require`, `revert`, or `throw` combined with an owner/sender comparison

---

Feature 4: `require_count`

- **Type:** Integer

- **What it checks:** Total number of `require()` statements in the contract.
- **Why it matters:** Honeypots often abuse `require()` to insert hidden conditions that block withdrawal. A high count relative to contract size can indicate excessive gating. Also useful for distinguishing well-guarded legitimate contracts from trap-heavy honeypots.
- **HoneyBadger type:** Hidden State, Hidden Transfer
- **Extraction method:** Regex (`require\s*\(`) or AST function call count

---

Feature 5: `has_selfdestruct`

- **Type:** Boolean
- **What it checks:** Does the contract contain a `selfdestruct()` (or deprecated `suicide()`) call?
- **Why it matters:** `selfdestruct` destroys the contract and sends all its ETH to a specified address. Honeypot owners can drain all victim funds and destroy the evidence in one call.
- **HoneyBadger type:** Hidden Transfer, Balance Disorder
- **Extraction method:** Regex or AST scan for `selfdestruct` / `suicide`

Feature 6: `has_withdrawal_function`

- **Type:** Boolean
- **What it checks:** Does the contract have a function whose name or logic resembles a withdrawal (e.g., `withdraw`, `claim`, `getBalance`, `cashOut`)?
- **Why it matters:** Honeypots advertise a withdrawal function to lure victims. Detecting its presence helps identify contracts that are designed around a

deposit-and-withdraw pattern — which is the setup for most honeypot traps.

- **HoneyBadger type:** All types (structural signal)
- **Extraction method:** AST — check `FunctionDefinition` names against a keyword list: `withdraw`, `claim`, `cashout`, `getfunds`, `retrieve`

Feature 7: `withdrawal_owner_only`

- **Type:** Boolean
- **What it checks:** Is the withdrawal function restricted to the contract owner (via `onlyOwner`, `msg.sender == owner`, or equivalent)?
- **Why it matters:** A withdrawal function that only the owner can call is the smoking gun — victims can deposit but only the deployer can withdraw. Combined with Feature 6, this is a strong honeypot signal.
- **HoneyBadger type:** Hidden Transfer, Balance Disorder
- **Extraction method:** AST — inside withdrawal function body, check for `require(msg.sender == owner)` or `onlyOwner` modifier

---

Feature 8: `uses_inline_assembly`

- **Type:** Boolean
- **What it checks:** Does the contract contain any `assembly { }` blocks?
- **Why it matters:** Inline assembly bypasses Solidity's safety checks and can hide logic that static analysis tools can't easily parse. Honeypot authors use it to conceal state manipulation, hidden jumps, or obfuscated withdrawal conditions.
- **HoneyBadger type:** Hidden State, Type Deduction Overflow

- **Extraction method:** Regex (`assembly\s*{`) or AST — look for `InlineAssembly` nodes

---

Feature 9: `state_changes_before_external_calls`

- **Type:** Integer
- **What it checks:** How many state variable assignments happen before any external call (e.g., `.call()`, `.transfer()`, `.send()`) in the same function?
- **Why it matters:** This is related to reentrancy-style patterns. In honeypots, state changes before external calls can be used to silently alter balances or flags before a victim's transaction completes — making the outcome different from what the victim expected.
- **HoneyBadger type:** Hidden State, Uninitialised Struct
- **Extraction method:** AST — within each function, count `StateVariableDeclarationStatement` or assignment nodes that appear before any `ExpressionStatement` containing `.call`/`.transfer`

Feature 10: `payable_to_withdraw_ratio`

- **Type:** Float
- **What it checks:** Number of `payable` functions divided by number of withdraw-like functions.
- **Why it matters:** A honeypot is designed to accept many deposits but block all withdrawals. A high ratio (many payable entry points, few or zero withdrawal paths) is a structural signal of a trap. A legitimate contract typically has a more balanced ratio.

- **Formula:** `count(payable functions) / max(count(withdraw-like functions), 1)` — use `max(..., 1)` to avoid division by zero
- **HoneyBadger type:** Balance Disorder, Hidden Transfer
- **Extraction method:** AST — count `FunctionDefinition` nodes with `payable` modifier; count functions whose name matches the withdrawal keyword list from Feature 6

Summary Table

#	Feature Name	Type	Honeypot Signal
1	`uses_tx_origin`	Boolean	Owner-only auth hiding
2	`has_fallback`	Boolean	Deposit trap presence
3	`fallback_reverts_non_owner`	Boolean	Silent deposit lock
4	`require_count`	Integer	Hidden condition density
5	`has_selfdestruct`	Boolean	Fund drain + destroy
6	`has_withdrawal_function`	Boolean	Withdrawal lure present
7	`withdrawal_owner_only`	Boolean	Withdrawal blocked for victims
8	`uses_inline_assembly`	Boolean	Hidden/obfuscated logic

#	Feature Name	Type	Honeypot Signal
9	`state_changes_before_external_calls`	Integer	State manipulation before payout
10	`payable_to_withdraw_ratio`	Float	Many deposits, no exit

Notes for Paper

- Features 1, 3, 5, 7, 8 are binary classifiers directly tied to known HoneyBadger deception types
- Features 4, 9, 10 are continuous/count features that give the Random Forest gradient signal
- Feature importance rankings (from `RandomForestClassifier.feature_importances_`) will be reported in Week 5 analysis
- Extraction tool: `solidity-parser` (Python) for AST; regex used as fallback for simpler string-level features